

AWS Assignment Solution

1. Explain the difference between AWS Regions, Availability Zones, and Edge Locations. Why is this important for data analysis and latency-sensitive applications.

AWS Regions : Region is a geographically distinct location which have multiple availability zones.

Availability Zones : Availability Zone is a physically isolated data center within a Region. Each Region has at least two availability zones.

Edge Locations : These are data centers specifically designed to deliver content faster to end users.

Importents:-

Reduced Latency : Placing compute and data resources in the closest Region or Edge Location minimizes delays in data access or streaming, essential for real-time analytics or dashboards.

High Availability and Fault Tolerance : Splitting workloads across multiple availability zones ensures analytics systems remain operational during infrastructure failures.

Faster Content Delivery : Edge Locations accelerate the delivery of dashboards, reports, and other static or pre-processed data to end-users globally.

2. Using the AWS CLI, list all available AWS regions. Share the command used and the output

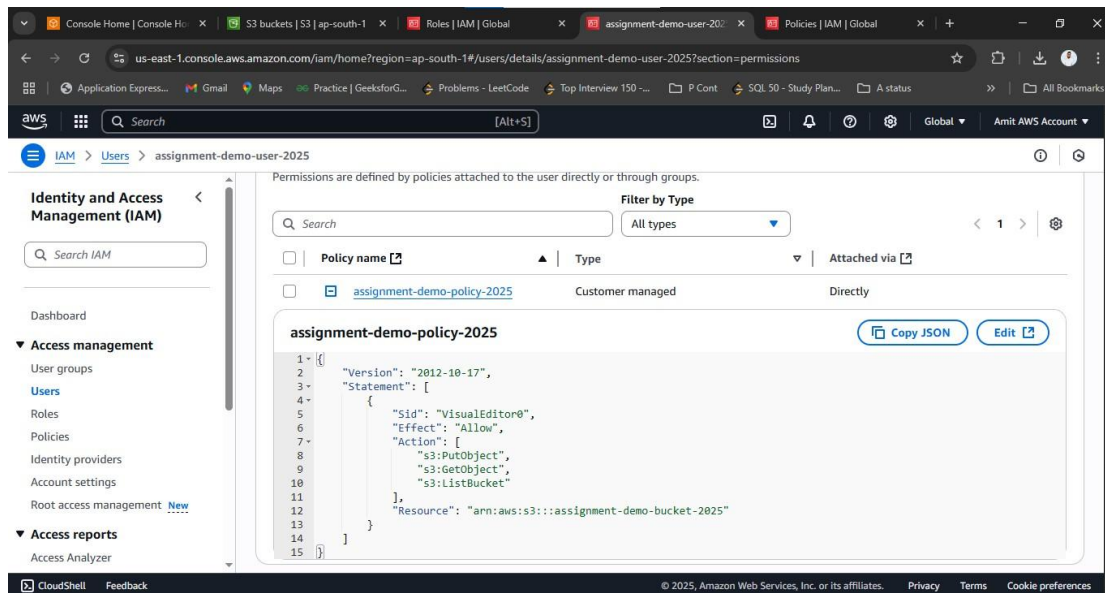
```
C:\WINDOWS\system32\cmd.exe - aws ec2 describe-regions --all-regions --query "Regions[*].RegionName" --output table
C:\Users\lenovo> aws configure
AWS Access Key ID [*****6YFD]: AKIAX5SBFTYUWGME2NFU
AWS Secret Access Key [*****HbBz]: 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
Default region name [ap-south-1]: ap-south-1
Default output format [json]: json

C:\Users\lenovo> aws ec2 describe-regions --all-regions --query "Regions[*].RegionName" --output table

DescribeRegions
+-----+
ap-south-2
ap-south-1
eu-south-1
eu-south-2
eu-central-1
il-central-1
sa-central-1
me-central-1
eu-central-1
eu-central-2
us-west-1
us-west-2
af-south-1
eu-north-1
eu-west-1
eu-west-2
eu-west-3
ap-northeast-1
ap-northeast-2
me-south-1
ap-northeast-3
sa-east-1
ap-east-1
ca-west-1
ap-southeast-1
ap-southeast-2
ap-southeast-3
More --

```

3. Create a new IAM user with least privilege access to Amazon S3. Share your attached policies (JSON or screenshot)



4. Compare different Amazon S3 storage (Standard, Intelligent-Tiering, Glacier). When should each be used in data analytics workflows?

S3 Standard

Use for: Frequently accessed datasets because of Low latency and high availability required during processing eg: Files used by ETL/ELT jobs or machine learning training

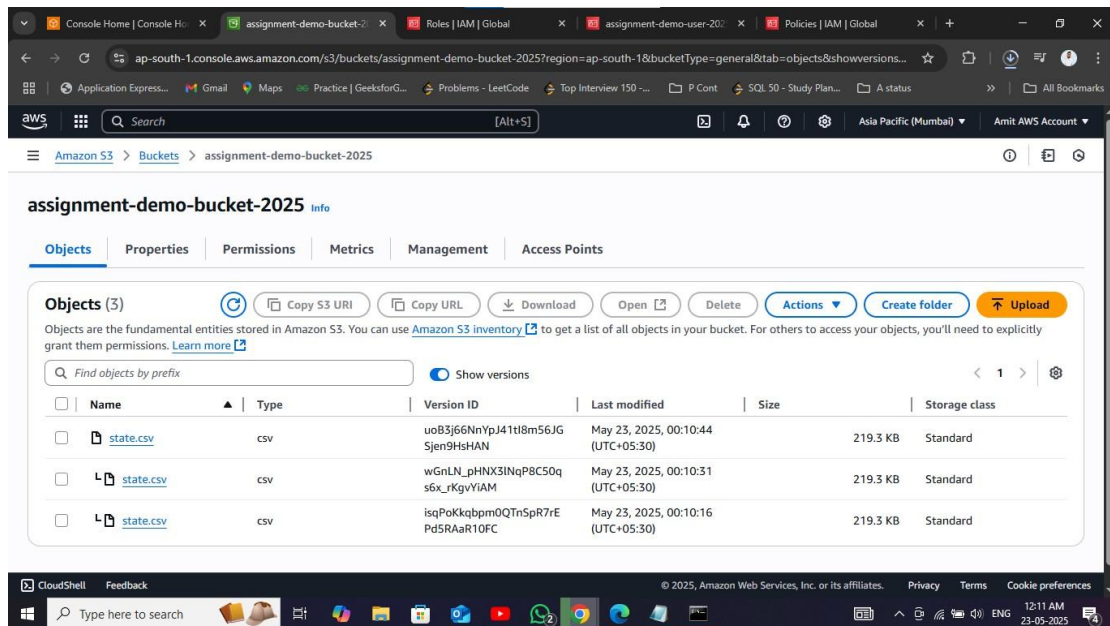
Intelligent-Tiering

Use for: Datasets with unpredictable access patterns because of Automatically moves objects to cheaper tiers if they aren't accessed, saving cost without performance sacrifice.

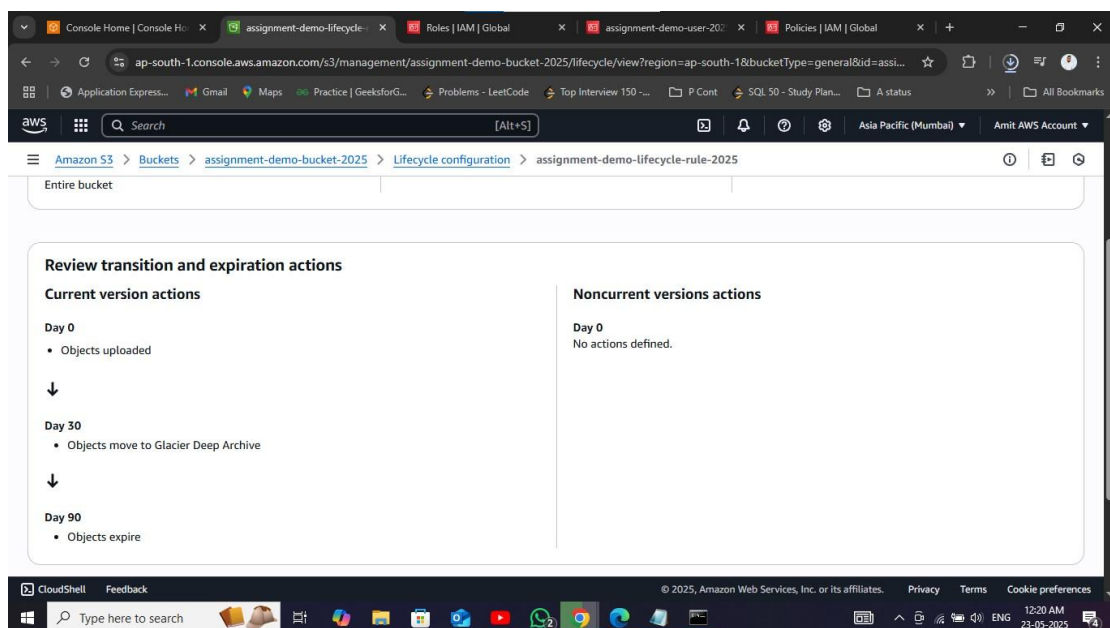
S3 Glacier / Glacier Deep Archive

Use for: Cold data — not used often but must be retained because of Very cheap, but requires restore time (minutes to hours), so not for real-time use.

5. Create an S3 bucket and upload a sample dataset (CSV or JSON). Enable versioning and show at least two versions of one file



6. Write and apply a lifecycle policy to move files to Glacier after 30 days and delete them after 90. Share the policy JSON or Screenshot.



7. Compare RDS, DynamoDB, and Redshift for use in different stages of a data pipeline. Give one use case for each

Amazon RDS is ideal for transactional workloads and structured data, making it suitable for applications like e-commerce platforms where relational data integrity is crucial.

Use Case: E-commerce Order Management

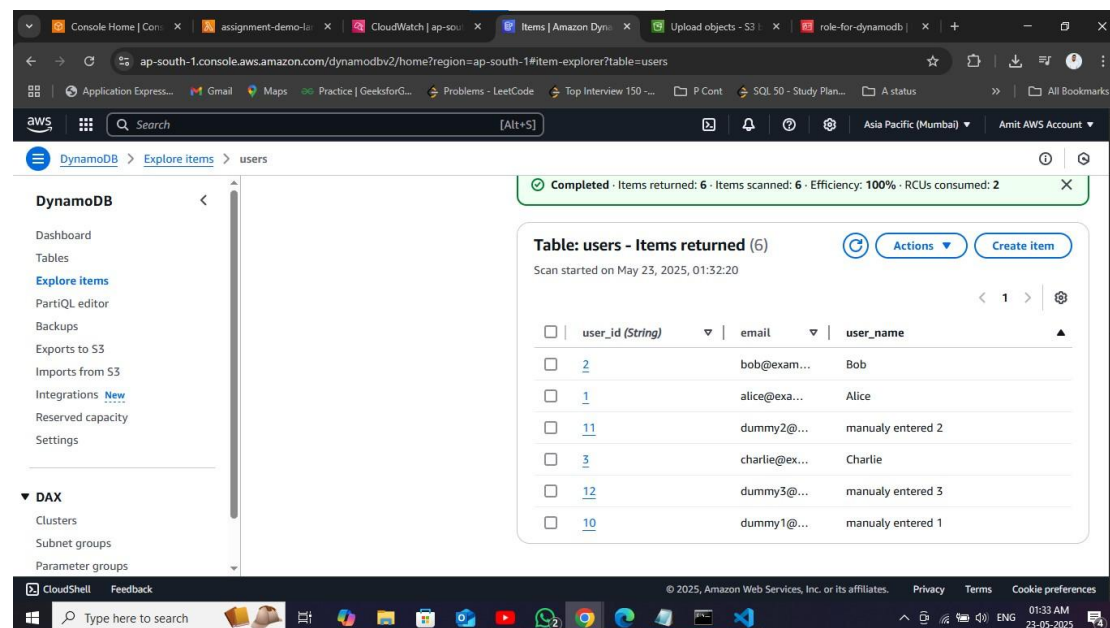
DynamoDB excels in handling high-velocity, unstructured data, making it perfect for real-time analytics in IoT applications.

Use Case: IoT Sensor Data Ingestion

Redshift is designed for large-scale data warehousing, ideal for business intelligence and analytics, such as aggregating sales data for reporting.

Use Case: Business Intelligence and Analytics

8. Create a DynamoDB table and insert 3 records manually. Then write a Lambda function that adds records when triggered by S3 uploads.



lambda function:-

```
import json, urllib, boto3, csv
```

```
# Connect to S3 and DynamoDB
```

```
s3 = boto3.resource('s3')
```

```
dynamodb = boto3.resource('dynamodb')
```

```
# Connect to the DynamoDB tables
```

```
usersTable = dynamodb.Table('users');
```

```
# This handler is run every time the Lambda function is triggered
```

```
def lambda_handler(event, context):
```

```
    # Show the incoming event in the debug log
```

```
    print("Event received by Lambda function: " + json.dumps(event, indent=2))
```

```
    # Get the bucket and object key from the event
```

```
    bucket = event['Records'][0]['s3']['bucket']['name']
```

```
    key = urllib.parse.unquote_plus(event['Records'][0]['s3']['object']['key'])
```

```

localFilename = '/tmp/users.txt'

# Download the file from S3 to the local filesystem
try:
    s3.meta.client.download_file(bucket, key, localFilename)
except Exception as e:
    print(e)
    print('Error getting object {} from bucket {}. Make sure they exist and your bucket
is in the same region as this function.'.format(key, bucket))
    raise e

# Read the users CSV file
with open(localFilename) as csvfile:
    reader = csv.DictReader(csvfile, delimiter=',')

# Read each row in the file
rowCount = 0
for row in reader:
    rowCount += 1

# Show the row in the debug log
print(row['user_id'], row['user_name'], row['email'])

try:
    # Insert Store, Item, and Count into the users table
    usersTable.put_item(
        Item={
            'user_id': row['user_id'],
            'user_name': row['user_name'],
            'email': row['email']}})

except Exception as e:
    print(e)
    print("Unable to insert data into DynamoDB table".format(e))

# Finished!
return "%d counts inserted" % rowCount

```

9. What is serverless computing? Discuss pros and cons of using AWS Lambda for data pipelines?

Serverless computing is a cloud computing model where we run applications without managing servers. We write code, deploy it, and the cloud provider (like AWS) automatically handles provisioning, scaling, and infrastructure.

Pros of Using AWS Lambda for Data Pipelines :-

No Server Management : Focus purely on writing transformation code. No provisioning or patching.

Cost-Efficient: Pay only per execution time (100ms granularity). Ideal for infrequent jobs.

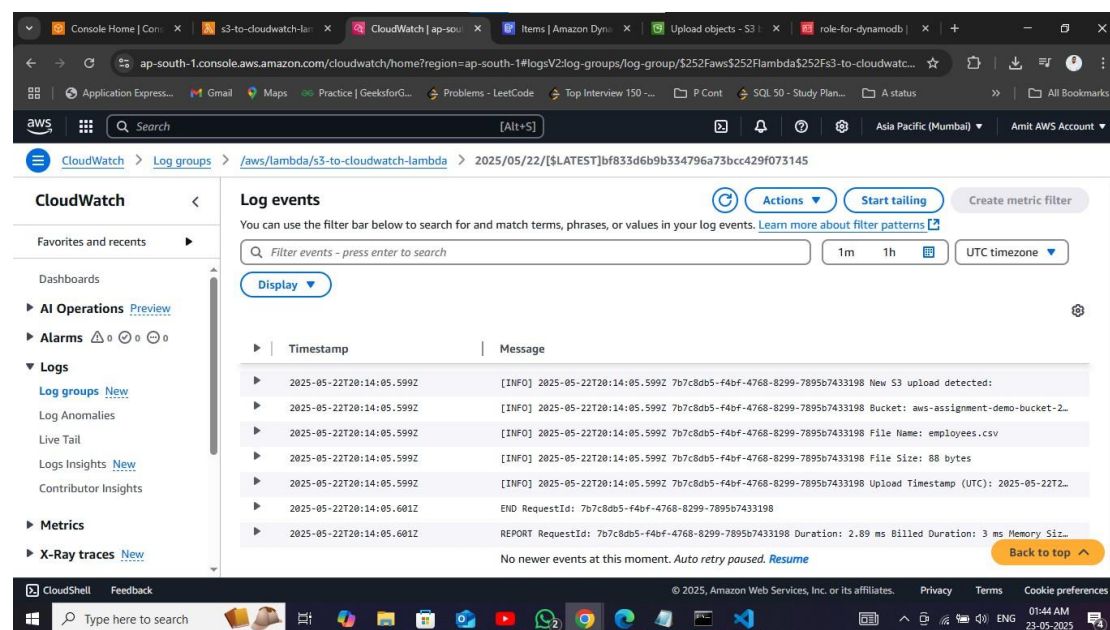
Scalable : Automatically scales based on workload—no manual intervention needed.

Cons of Using AWS Lambda for Data Pipelines :-

Execution Time Limit : Max 15 minutes per invocation which is not suitable for long-running batch jobs.

Dependency Management : Complex for functions needing large libraries (e.g., Pandas, NumPy).

10. Create a Lambda function triggered by S3 uploads that logs file name, size, and timestamp to Cloudwatch. Share code and a log screenshot.



```
import json
import logging
from datetime import datetime
```

```
# Set up logging to CloudWatch
```

```
logger = logging.getLogger()
logger.setLevel(logging.INFO)
```

```
def lambda_handler(event, context):
    for record in event['Records']:
        s3_info = record['s3']
        bucket_name = s3_info['bucket']['name']
        object_key = s3_info['object']['key']
        file_size = s3_info['object'].get('size', 'Unknown')
        upload_time = datetime.utcnow().isoformat()
```

```
    logger.info(f'New S3 upload detected:')
    logger.info(f'Bucket: {bucket_name}')
    logger.info(f'File Name: {object_key}')
```



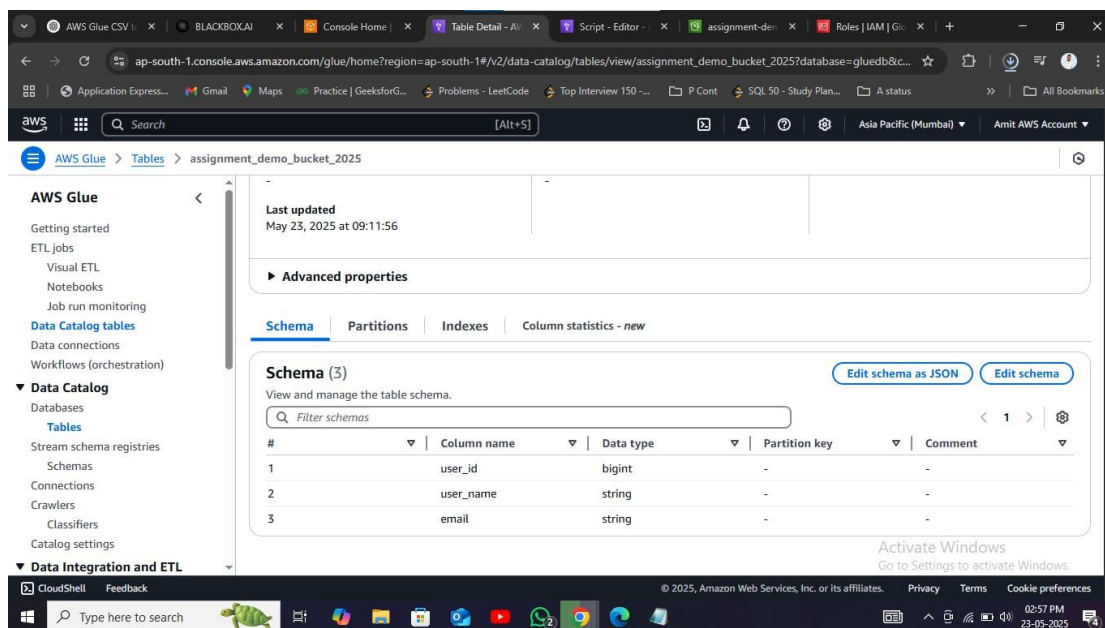
```

logger.info(f'File Size: {file_size} bytes")
logger.info(f'Upload Timestamp (UTC): {upload_time}")

return {
    'statusCode': 200,
    'body': json.dumps('S3 upload log recorded successfully.')
}

```

11. Use AWS Glue to crawl your S3 dataset, create a Data Catalog table, and run a Glue job to convert CSV data to parquet. Share job code and output location.



```

import sys
from awsglue.transforms import *
from awsglue.utils import getResolvedOptions
from pyspark.context import SparkContext
from awsglue.context import GlueContext
from awsglue.job import Job

args = getResolvedOptions(sys.argv, ['JOB_NAME'])
sc = SparkContext()
glueContext = GlueContext(sc)
spark = glueContext.spark_session
job = Job(glueContext)
job.init(args['JOB_NAME'], args)

```

```

# Script generated for node Amazon S3
AmazonS3_node1747991885789 =
glueContext.create_dynamic_frame.from_options(format_options={"quoteChar": "\"",
"withHeader": True, "separator": ",", "optimizePerformance": False},
connection_type="s3", format="csv", connection_options={"paths":

```

```
[ "s3://assignment-demo-bucket-2025/users.csv"], "recurse": True},
transformation_ctx="AmazonS3_node1747991885789")

# Script generated for node Rename Field
RenameField_node1747991941282 =
RenameField.apply(frame=AmazonS3_node1747991885789, old_name="email",
new_name="e-mail", transformation_ctx="RenameField_node1747991941282")

# Script generated for node AWS Glue Data Catalog
AWSGlueDataCatalog_node1747991982360 =
glueContext.write_dynamic_frame.from_catalog(frame=RenameField_node17479919
41282, database="gluedb", table_name="assignment_demo_bucket_2025",
transformation_ctx="AWSGlueDataCatalog_node1747991982360")

job.commit()
```

12. Explain the difference between Kinesis Data Streams, Kinesis Firehose, and Kinesis Data Analytics. Provide a real-world example of how each would be used.

Kinesis Data Streams : A real-time data streaming service.

Streams data into shards where producers write and consumers read data.

Allows custom processing using AWS Lambda, EC2 apps, etc.

Use Case:

Real-time fraud detection in banking

A bank streams every transaction through Kinesis Data Streams. A Lambda function reads the stream and analyzes transactions for patterns of fraud within milliseconds.

Kinesis Data Firehose :- A fully managed service that automatically loads streaming data to destinations like:

Amazon S3

Amazon Redshift

Amazon OpenSearch

Use Case:

Website clickstream analytics

An e-commerce site sends real-time clickstream data to Kinesis

Firehose, which delivers it to Amazon S3. From there, Redshift runs hourly analytics to understand user behavior.

Kinesis Data Analytics :-

A service to analyze streaming data in real time using SQL, Apache Flink, or Java applications.

Works with Kinesis Data Streams, Firehose, or Kafka as input.

Use Case:

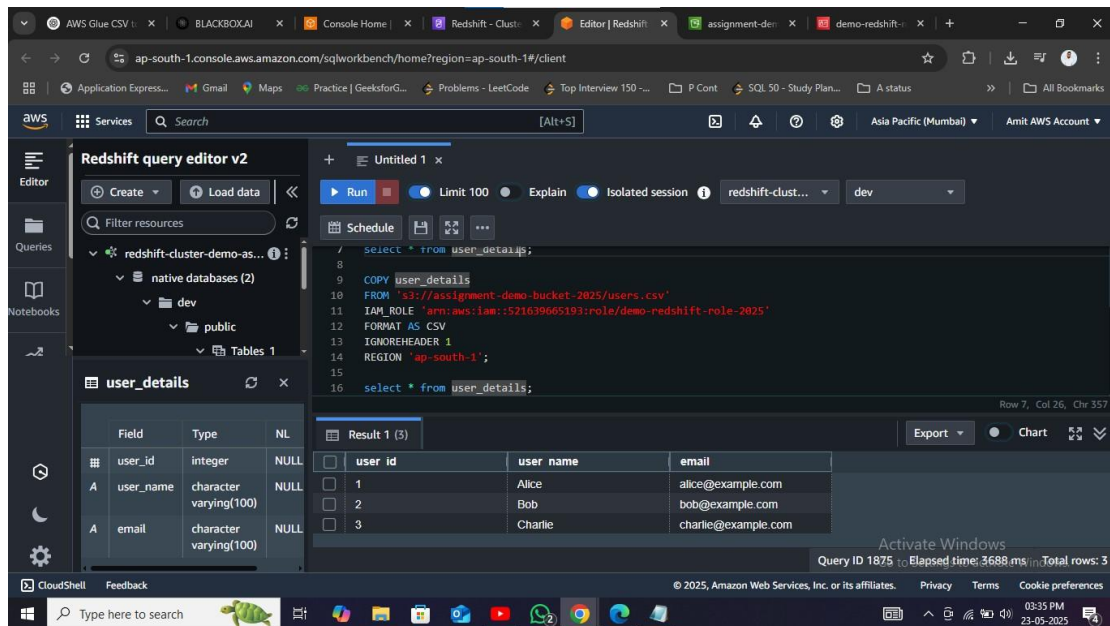
IoT sensor monitoring

A factory sends real-time temperature and vibration sensor data to Kinesis. Kinesis Data Analytics applies SQL queries to detect when average temperature over 10 seconds exceeds a threshold, and triggers an alert via Lambda.

13. What is columnar storage and how does it benefit Redshift performance for analytics workloads?

Columnar storage organizes data by columns rather than rows, allowing for efficient data retrieval and processing. In Redshift, this structure enhances performance for analytics workloads by reducing disk I/O and enabling faster query execution, as only the necessary columns are accessed for specific queries.

14. Load a CSV file from S3 into Redshift using the COPY command. Share table schema, command used, and sample output from a query.



create TABLE user_details

(

user_id int,

user_name varchar(100),

email VARCHAR(100)

);

select * from user_details;

COPY user_details

FROM 's3://assignment-demo-bucket-2025/users.csv'

IAM_ROLE 'arn:aws:iam::521639665193:role/demo-redshift-role-2025'

FORMAT AS CSV

IGNOREHEADER 1

REGION 'ap-south-1';

select * from user_details;

15. What is the role of the AWS Glue Data Catalog in Athena? How does schema-on-read work?

The AWS Glue Data Catalog serves as a centralized repository for metadata, storing information about tables, columns, and data locations used by Athena. Schema-on-read allows Athena to interpret the data structure at query time, enabling flexibility in handling various data formats without needing to predefine schemas.

16. Create an Athena table from S3 data using Glue Catalog. Run a query and share the SQL + result screenshot.

The screenshot shows the Amazon Athena console interface. On the left, the 'Data source' is set to 'AwsDataCatalog', 'Catalogue' is 'None', and 'Database' is 'athenadb'. The 'Tables and views' section shows a table named 'assignment_demo_bucket_2025'. The main query editor contains the SQL: `select user_name,email from assignment_demo_bucket_2025 where user_id = 2;`. The query has been executed successfully, showing a 'Completed' status with a 'Time in queue' of 112 ms, 'Run time' of 989 ms, and 'Data scanned' of 0.10 KB. The results table has two columns: 'user_name' and 'email', with one row containing 'Bob' and 'bob@example.com'.

Query:-

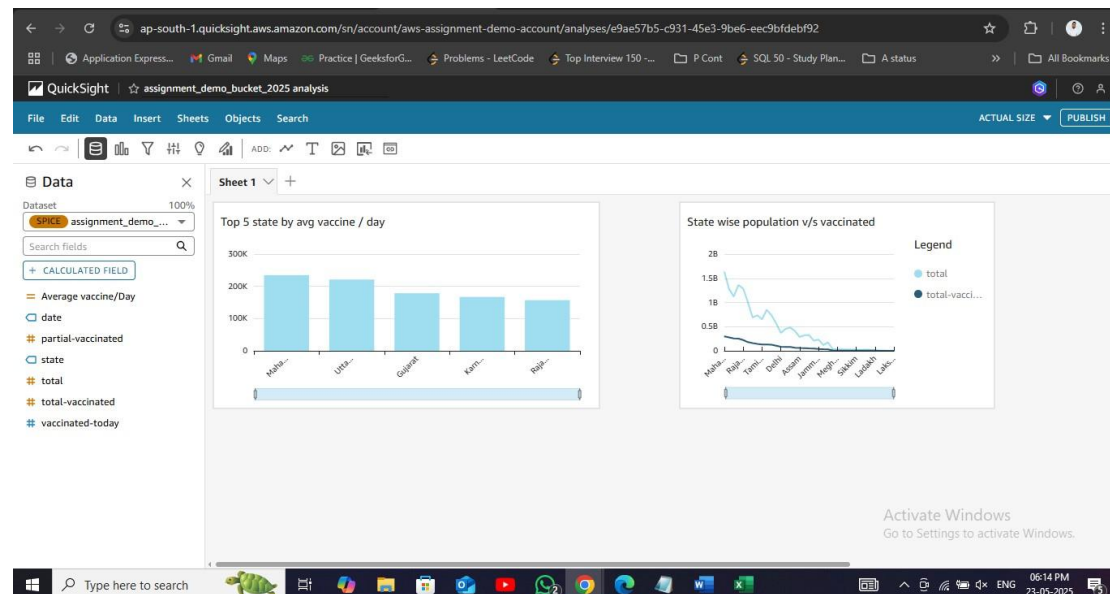
```
select user_name,email from assignment_demo_bucket_2025 where user_id = 2;
```

17. Describe how Amazon QuickSight supports business intelligence in a serverless data architecture. What are SPICE and embedded dashboards.

Amazon QuickSight supports business intelligence in a serverless architecture by providing scalable data visualization and analytics without the need for infrastructure management. SPICE which is Super-fast, Parallel, In-memory Calculation Engine,

enhances performance by allowing fast data retrieval, while embedded dashboards enable seamless integration of analytics into applications for interactive insights.

18. Connect QuickSight to Athena or Redshift and build a dashboard with at least one calculated field and one filter. Share a screenshot of your final dashboard.



19. Explain how AWS CloudWatch and CloudTrail differ. IN a data analytics pipeline, what role does each play in monitoring, auditing, and troubleshooting.

CloudWatch: Primarily a monitoring service that tracks the performance and health of AWS resources and applications in real-time.

CloudTrail: Focuses on auditing and compliance by logging API calls and user activities within the AWS environment.

Roles in a Data Analytics Pipeline:-

Monitoring:

CloudWatch: Continuously monitors the performance of data processing resources, such as EC2 instances and databases, allowing for real-time alerts and dashboards to visualize operational metrics.

CloudTrail: While not a monitoring tool in the traditional sense, it provides insights into user activities and API calls that can indicate performance issues or unauthorized access.

Auditing:

CloudWatch: Offers logs that can be analyzed for operational insights but is not primarily designed for auditing purposes.

CloudTrail: Essential for auditing, it logs every API call made in the AWS account, providing a comprehensive history of actions taken, which is crucial for compliance and security reviews.

Troubleshooting:

CloudWatch: Facilitates troubleshooting by providing metrics and logs that help identify performance bottlenecks or failures in real-time.

CloudTrail: Aids in troubleshooting security incidents by allowing administrators to trace back actions taken by users or services, helping to identify the source of issues or breaches.

20. Describe a complete end-to-end data analytics pipeline using AWS services. Include services for data ingestion, storage, transformation, querying, and visualization. (Example: S3 → Lambda → Glue → Quicksight), Explain why you would choose each service for the stage it's used in.

End-to-End AWS Data Analytics Pipeline :-

Use Case: An e-commerce company analyzes user clickstream and transaction data to optimize marketing and product placement.

Pipeline Flow :-

Amazon Kinesis → AWS Lambda → Amazon S3 → AWS Glue → Amazon Athena → Amazon QuickSight

Data Ingestion – Amazon Kinesis Data Streams

Why: Real-time ingestion of user clickstream and transaction events from websites, mobile apps, or IoT devices. **How:** Applications send JSON-formatted event data to Kinesis Data Streams. **Benefit:** Handles high-throughput, low-latency streaming data with auto-scaling.

Real-Time Processing – AWS Lambda

Why: Perform real-time transformation and filtering of streaming data before it hits storage. How: Lambda is triggered by Kinesis; transforms records (e.g., flattens JSON, filters bots). Benefit: Serverless, event-driven compute with sub-second response time and low cost.

Data Storage – Amazon S3

Why: Durable, cost-effective storage for both raw and processed data (data lake). How: Lambda writes transformed data into partitioned folders by date in S3. Benefit: Scalable, highly available, supports multiple formats (CSV, Parquet, JSON).

Data Catalog & ETL – AWS Glue

Why: Manage metadata and perform ETL (Extract, Transform, Load). How: Glue Crawlers scan S3 and create/update tables in the Glue Data Catalog. Glue Jobs convert raw data to columnar formats (e.g., JSON → Parquet). Benefit: Serverless, integrated with S3 and Athena, supports schema-on-read.

Querying – Amazon Athena

Why: Interactive querying using standard SQL on data stored in S3. How: Analysts query Glue Data Catalog tables with Athena to generate summaries, joins, and filters. Benefit: Serverless, pay-per-query, supports ad-hoc exploration and dashboards.

Visualization – Amazon QuickSight

Why: Build dashboards and reports from Athena queries and Glue tables. How: Connect QuickSight to Athena via the Glue Data Catalog, create visualizations (e.g., revenue by region). Benefit: Scalable BI tool, supports ML insights, native AWS integration.